

Issues UI v0.1.4

Graph Visualisation & Architectural Improvements

IMPLEMENTATION BRIEF

Status: Draft for Review

Target Release: 01 Feb 2026

From: UI/Frontend Claude Code Session

To: Backend Team / Review

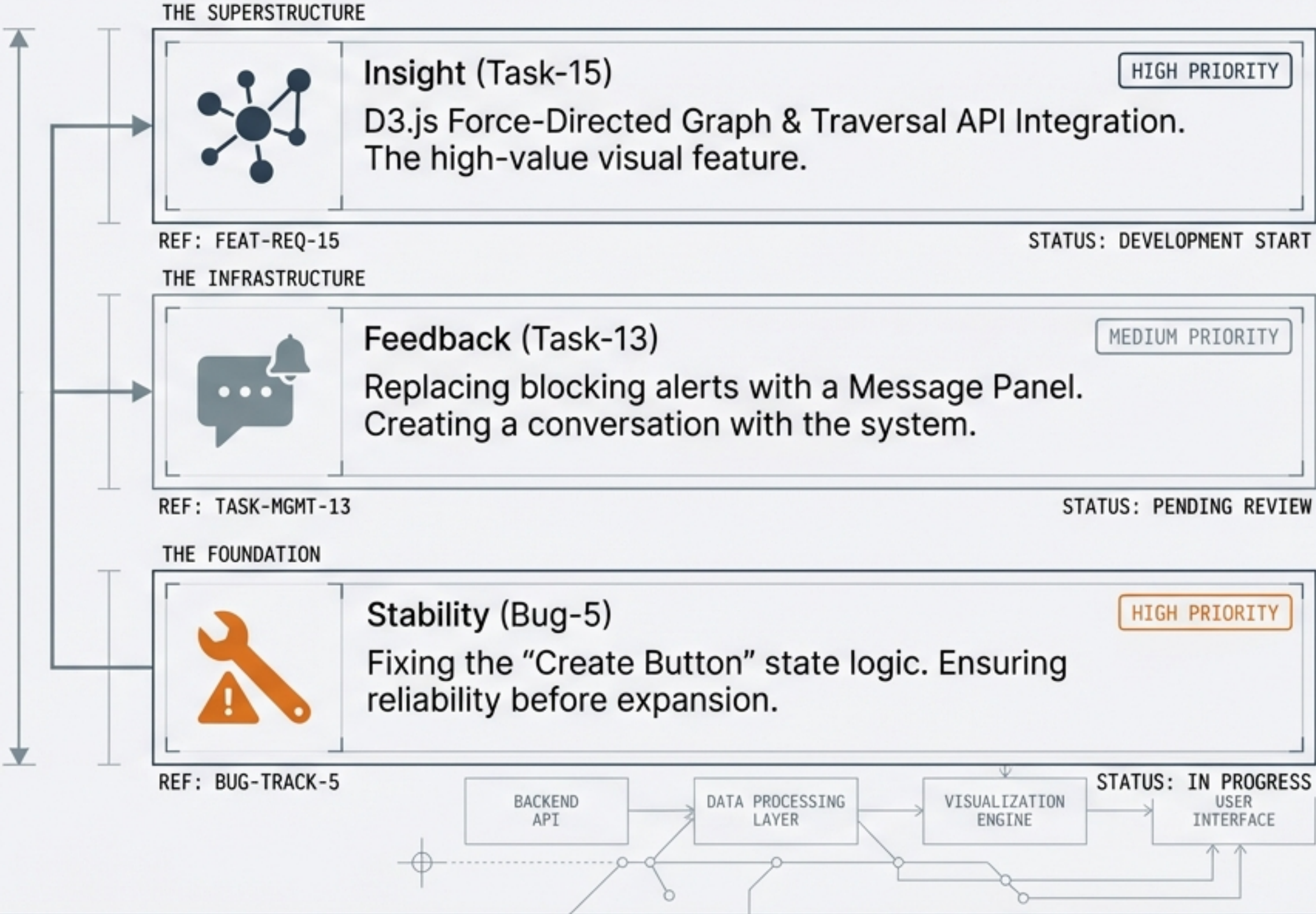
Theme: From Backend Enablement (v0.2.20) to Frontend Insight

STRATEGIC SCOPE: FUNCTIONAL FRICTION TO INTERACTIVE INSIGHT

IMPLEMENTATION BLUEPRINT FOR V0.1.4

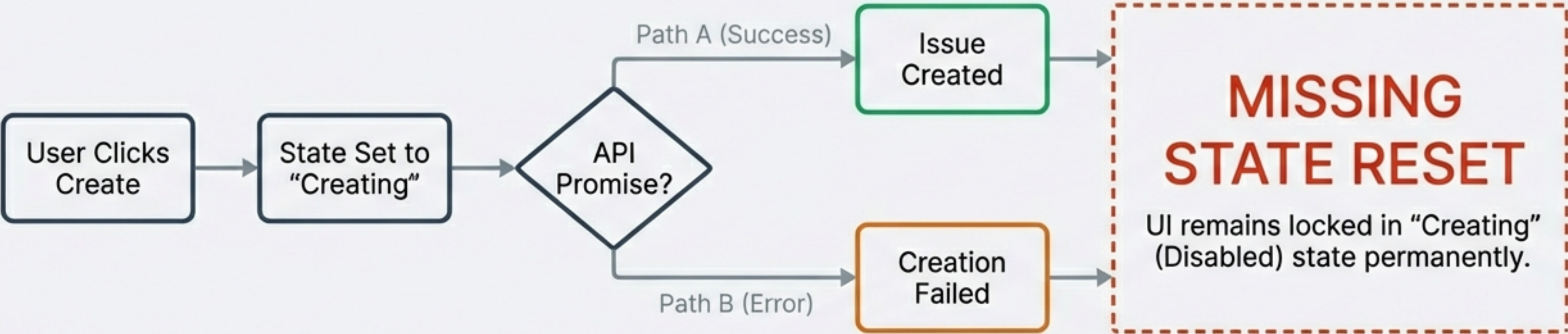
CONTEXT

Following the backend v0.2.20 unblock of the traversal API, v0.1.4 introduces the visual superstructure (Graph) while shoring up the foundation (Bug fixes). We are moving from fixing friction to enabling exploration.



STABILISING THE CORE: THE CREATE LOGIC FIX (BUG-5)

PROBLEM DIAGNOSIS & RESOLUTION



File: create-node-modal.js

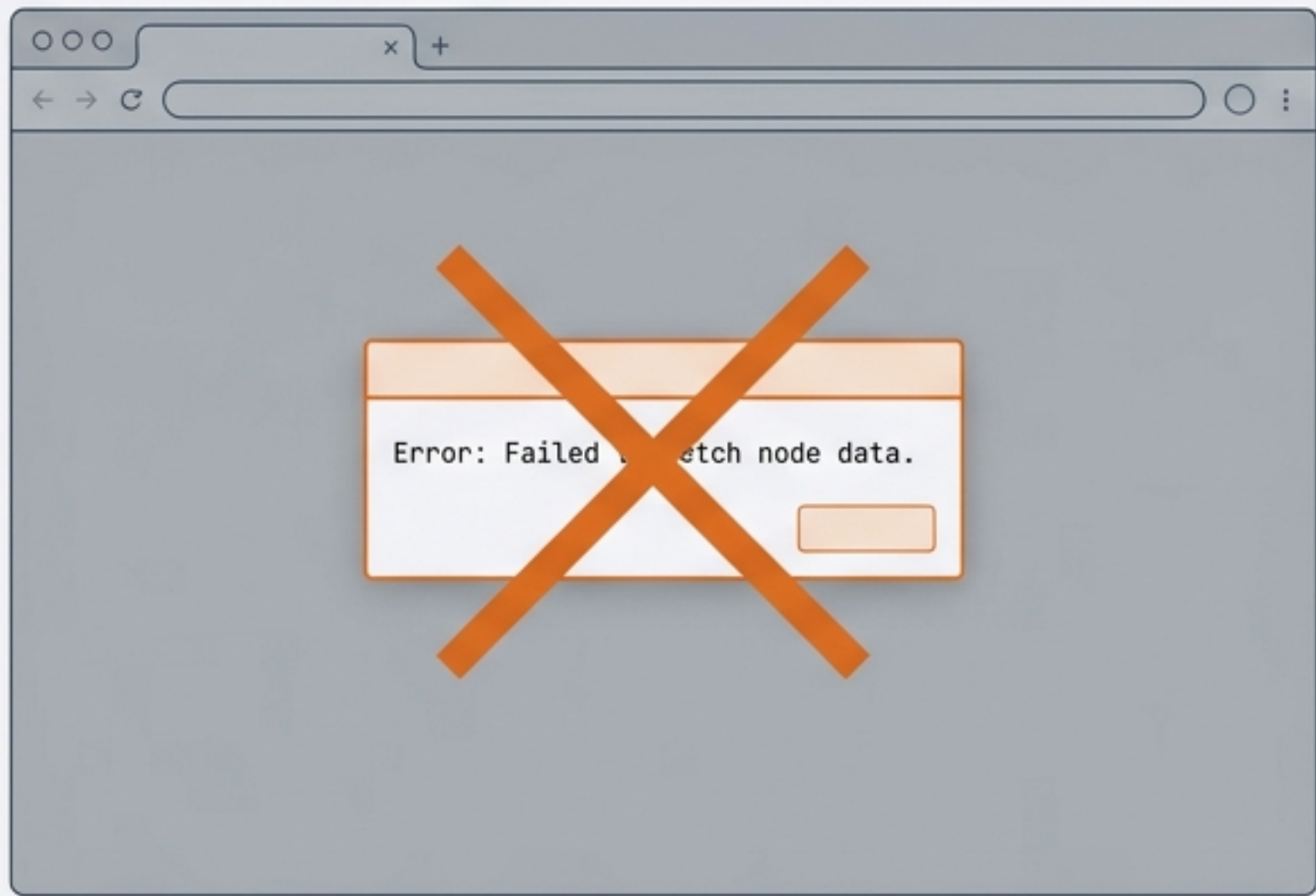
```
try {
  await api.createNode(data);
  this.close();
} catch (error) {
  this.showError(error);
} finally {
  this.isCreating = false; // <--- The Fix: Enforce reset
}
```

Scope also extends to kanban-board.js to ensure external buttons re-enable.

SYSTEM FEEDBACK ARCHITECTURE (TASK-13)

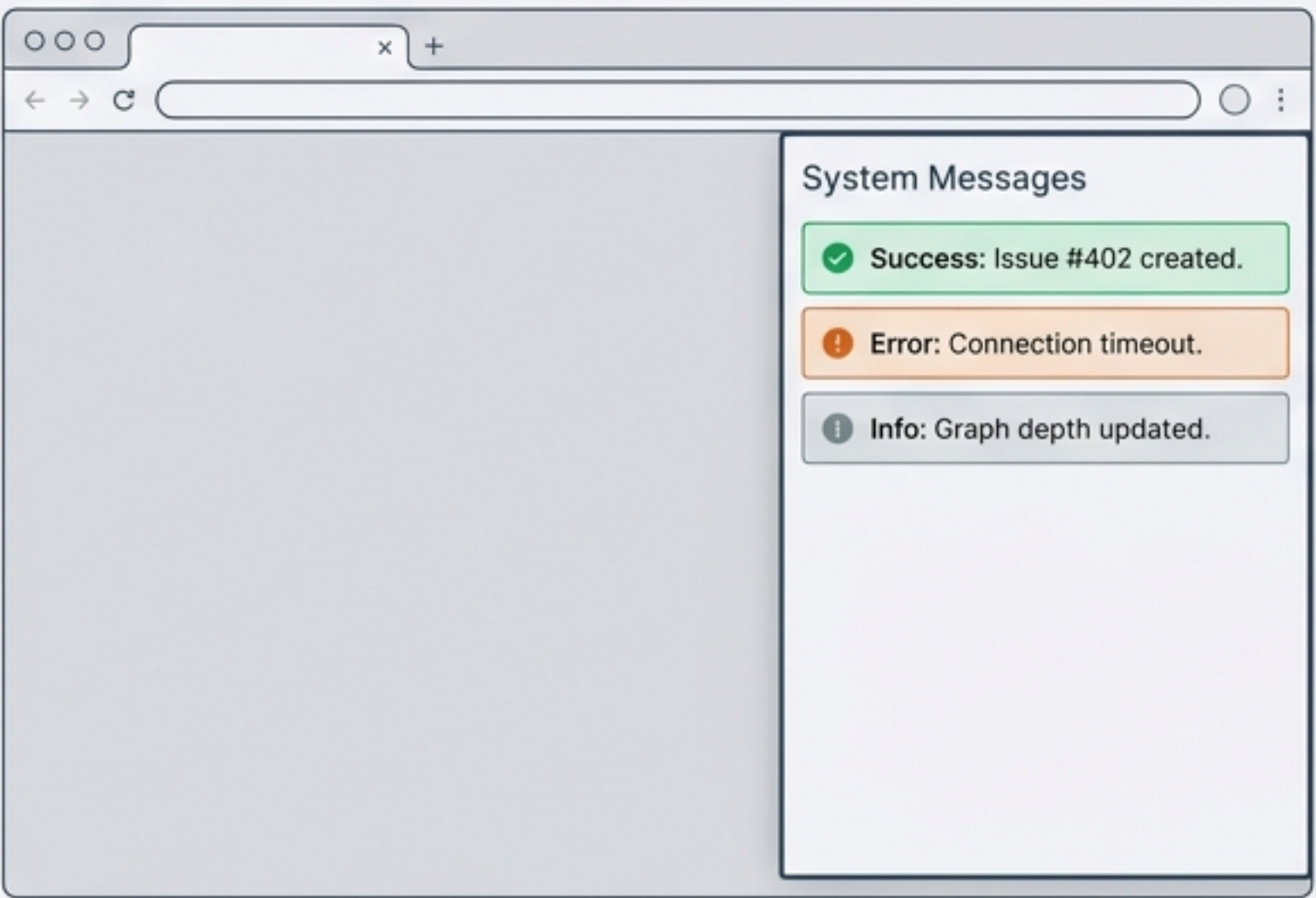
Moving from Blocking Interruptions to Persistent History

OLD WAY: Blocking



Blocks user interaction. No history. Ephemeral.

NEW WAY: Non-blocking



Persistent log. Non-intrusive. Collapsible.

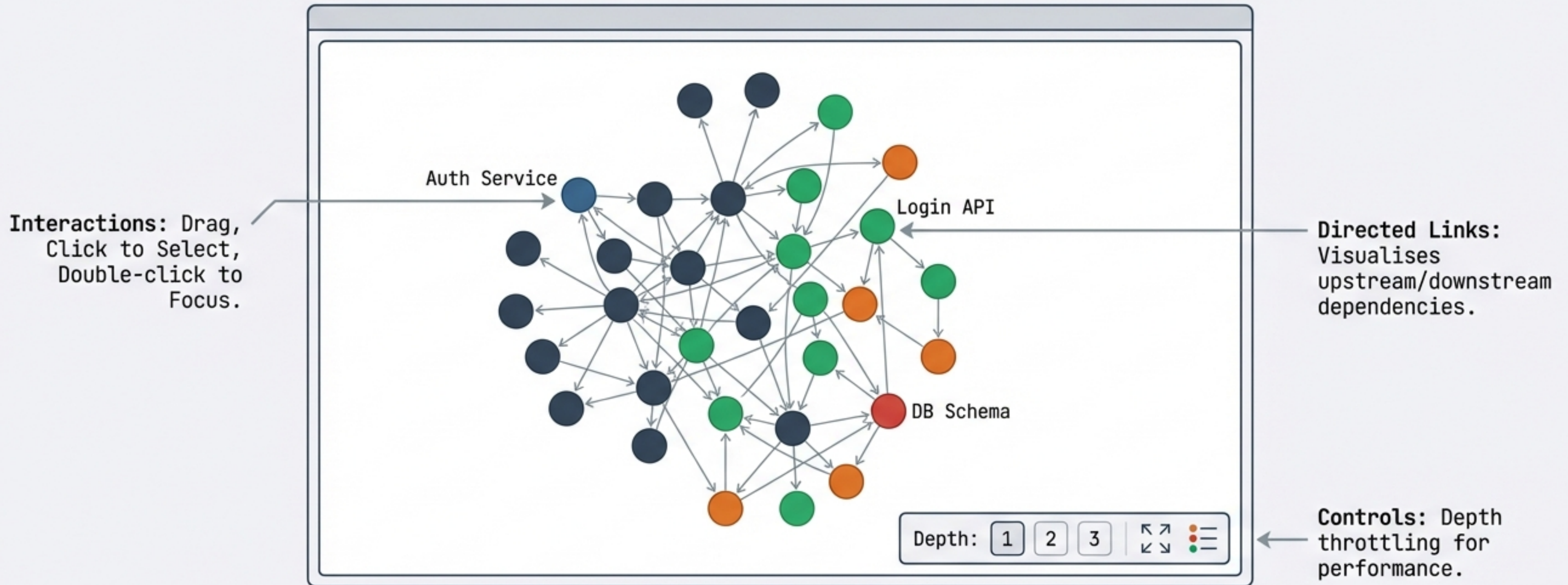
Component:
messages-panel.js

Service:
messages-service.js

Method:
EventBus Auto-subscription

VISUALISING COMPLEXITY: THE FORCE-DIRECTED GRAPH (TASK-15)

Enabling dependency exploration via D3.js



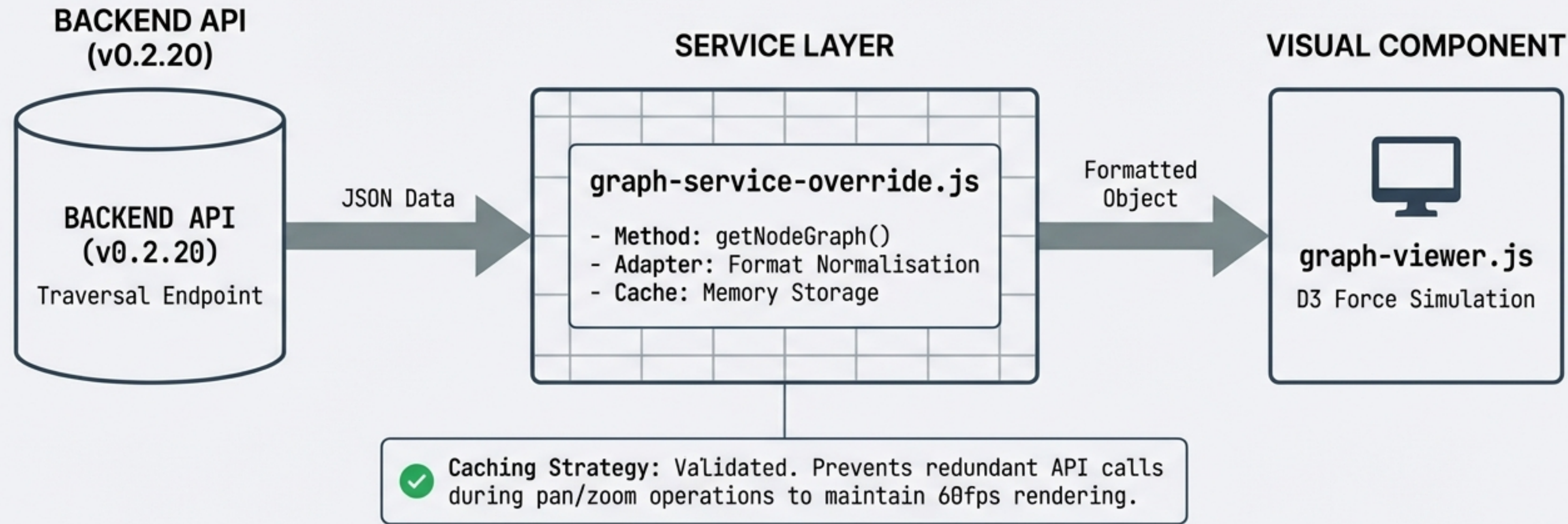
Core Tech: D3.js v7

Renderer: SVG/Canvas

Goal: Interactive Insight

THE DATA PIPELINE & API INTEGRATION (TASK-14)

From JSON Response to Visual Component



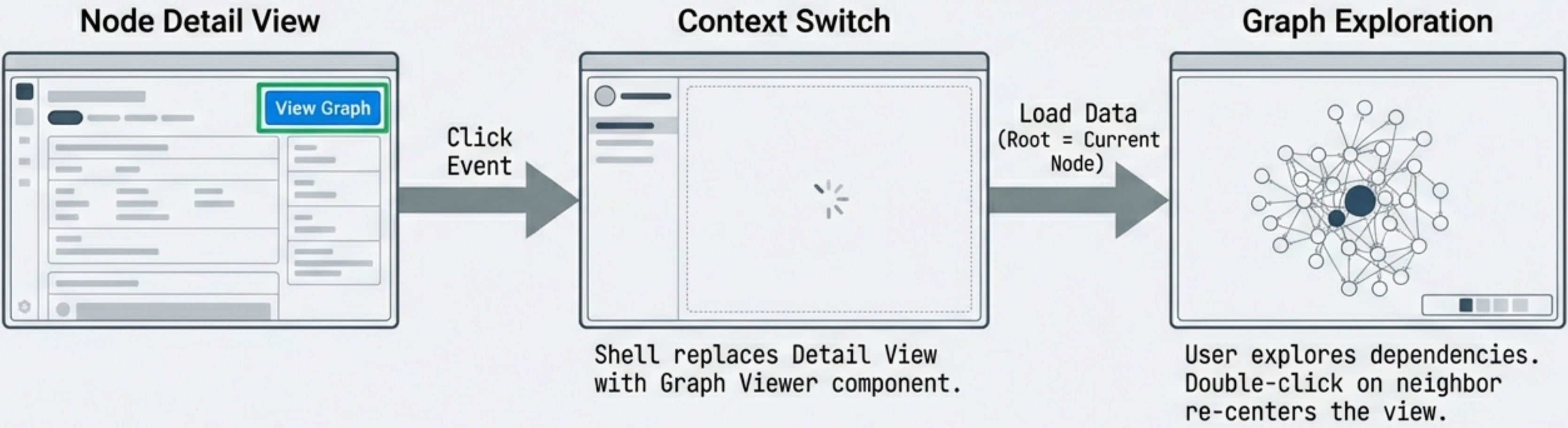
Component:
graph-viewer.js

Service:
graph-service-override.js

Method:
getNodeGraph()

USER NAVIGATION & ENTRY POINTS (TASK-16)

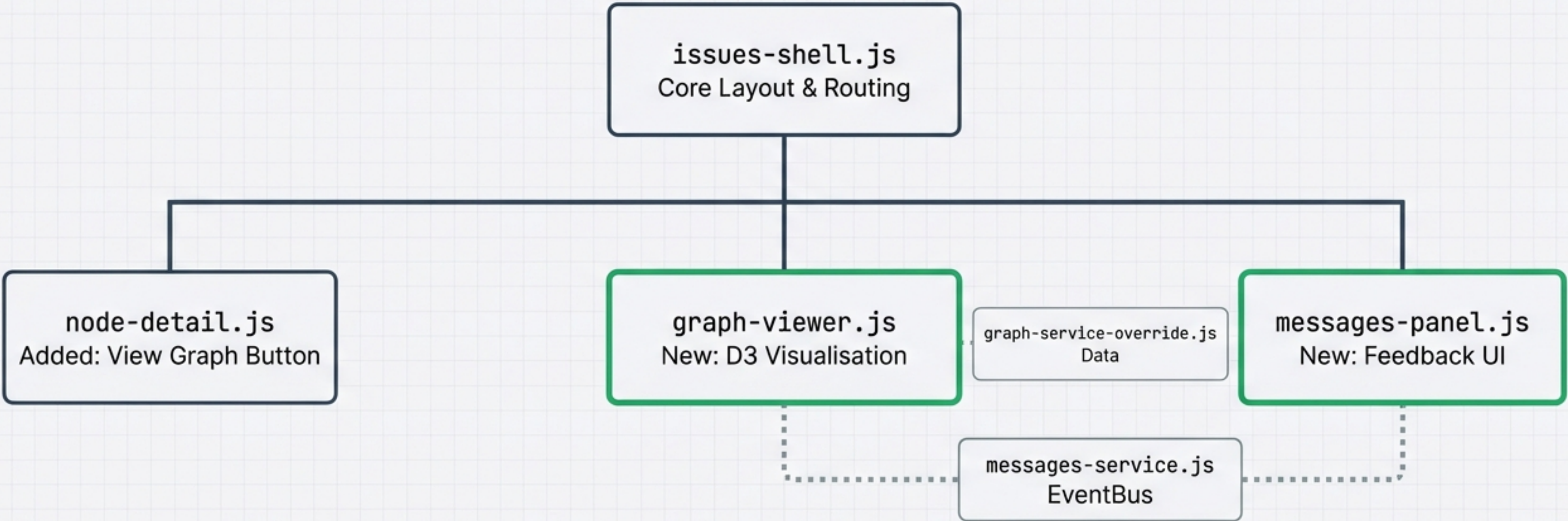
Context-Aware Navigation Logic



Design Decision: Main Content Navigation vs. Modal.
Selected: Main Content. Maximises screen real estate for complex diagrams and maintains consistency with existing single-content-area pattern.

INTEGRATION ARCHITECTURE & FILE STRUCTURE

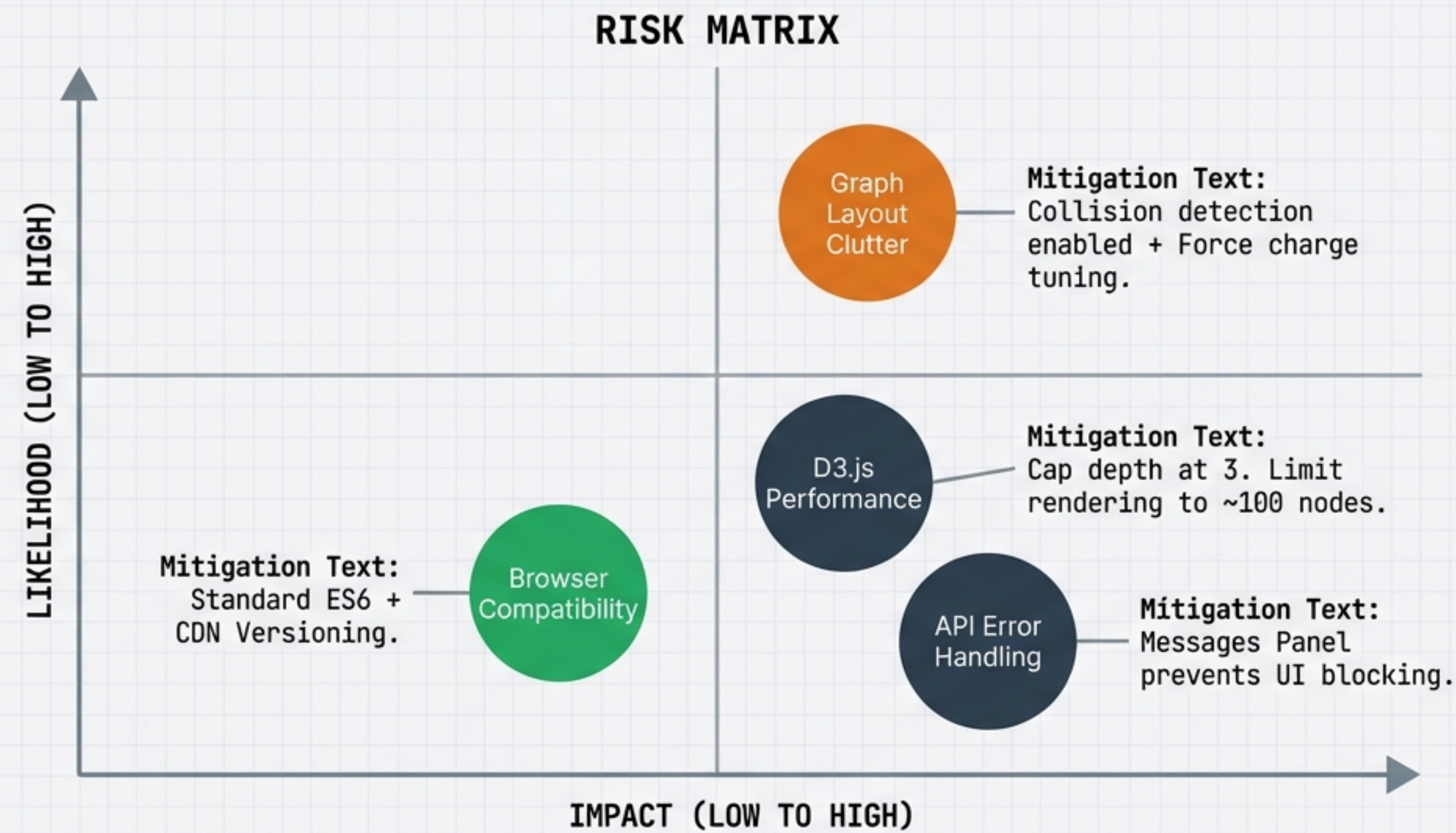
Component Dependency Tree



Key: Solid lines indicate Direct Child Components. Dotted lines indicate Service/Data Injection. ————— ······

RISK ASSESSMENT & MITIGATION STRATEGY

Proactive Technical Defence



SUMMARY:

Primary technical risk is client-side rendering performance.

Mitigation is strict data throttling at the service layer.

QUALITY ASSURANCE STRATEGY

Testing Pillars for v0.1.4

STABILITY (Bug-5)

- ☒ Create button resets state after first issue.
- ☒ UI recovers from API creation failures.
- ☒ Modal cancellation clears loading state.

FUNCTIONAL (Task-15)

- ☒ getNodeGraph returns valid JSON structure.
- ☒ Nodes and Links render with correct types.
- ☒ Pan, Zoom, and Drag interactions are smooth.
- ☒ Depth changes trigger data refresh.

UI & FEEDBACK (Task-13)

- ☒ Error messages auto-expand the side panel.
- ☒ Success messages auto-dismiss or log quietly.
- ☒ 'View Graph' button present in Node Detail header.

Definition of Done: All P0/P1 tests pass. Zero blocking console errors.

KEY DECISION POINTS & RECOMMENDATIONS

Resolving Open Architectural Questions

GRAPH NAVIGATION

Question: Modal Overlay vs. Main Content?

**RECOMMENDATION:
MAIN CONTENT**

Reason: Consistent with single-window UX. Better space for analysis.



PANEL LOCATION

Question: Right Sidebar vs. Bottom Bar?

**RECOMMENDATION:
RIGHT SIDEBAR**

Reason: Mimics Debug Panel pattern. Toggleable from header.



DEFAULT DEPTH

Question: Depth 1 or 2?

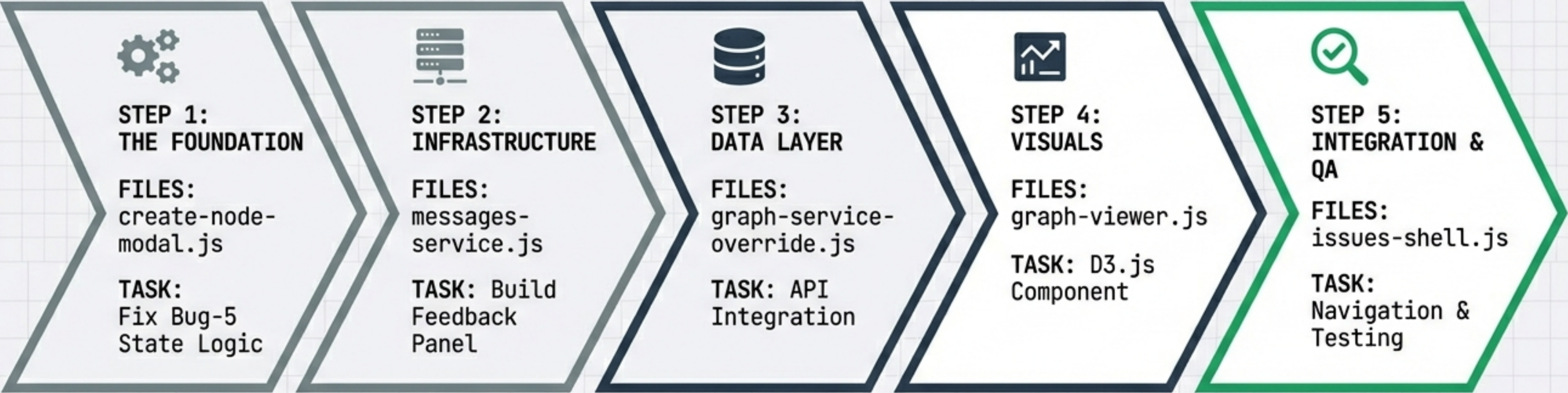
**RECOMMENDATION:
DEPTH 1**

Reason: Prioritise initial load performance. User can manually expand.



IMPLEMENTATION ROADMAP

Execution Order for v0.1.4



READY TO EXECUTE.